Shielded Payments for Decentralized Cloud Services: Anonymous Pay-Per-Use via Cross-Chain Shielded Pools, Prepaid Credits, and Rate-Limiting Nullifiers

Drew Stone
drew@tangle.tools
Tangle Network

March 2026

Abstract

We present a privacy-preserving payment system for decentralized cloud services—LLM inference, image generation, compute—where users pay anonymously and operators serve requests without learning the payer's identity. The system composes three layers: (1) a cross-chain UTXO-based shielded pool [1] with variable-amount JoinSplit transactions and multi-stablecoin wrapping, (2) a stateless gateway contract that bridges anonymous partial withdrawals to Tangle's Blueprint service lifecycle, and (3) a prepaid credit scheme where a single zero-knowledge proof funds an account from which many jobs are authorized via EIP-712 signatures. We describe the deployed system (Credit Mode) and a planned upgrade to per-request unlinkability via Rate-Limiting Nullifiers [3] composed with the existing audited VAnchor circuit (RLN Mode). The Credit Mode construction requires no modifications to Tangle's audited core contracts and introduces under 1,000 lines of new Solidity.

Contents

1 Introduction **1**
1.1 Contributions . . . 2

2 Preliminaries **2**

3 System Architecture (Credit Mode) **2**
3.1 UTXO Model and Partial Withdrawals . . . 2
3.2 Shielded Gateway . . . 3
3.3 Shielded Credits . . . 3

4 Privacy Analysis **4**
4.1 Operator Knowledge Boundary . . . 4
4.2 Unlinkability Across Sessions . . . 4
4.3 Limitations of Credit Mode . . . 5

5 Per-Request Unlinkability via Rate-Limiting Nullifiers (RLN Mode) **5**
5.1 How It Works (Intuition) . . . 5
5.2 Formal Definitions . . . 7
5.3 Composed Circuit . . . 7
5.4 Protocol . . . 9
5.5 Refund Accumulation . . . 9
5.6 Proof Pre-Generation and Parallelism . . . 10

5.7 Settlement Contract 10
5.8 Privacy Comparison 10

6 Use Cases **11**

7 Cross-Chain Architecture **11**

8 Batch Verification via SP1 zkVM **12**

9 Cost Analysis **12**

10 Implementation **12**

1 Introduction

Every major AI inference provider—OpenAI, Anthropic, Google—requires account creation, identity verification, and traceable payment. A developer querying an LLM for code review, legal analysis, or medical information creates a permanent record linking their identity to every prompt. Decentralized alternatives inherit the same problem: on-chain payments link wallet addresses to service consumption with full public auditability.

This is not hypothetical. Inference providers can profile users by query content, timing, and volume. Employers can monitor which employees use AI assistance. Competitors can observe which organizations are training custom models. The payment trail is the surveillance vector.

Buterin and Crapis [2] proposed *ZK API Usage Credits*: deposit into a smart contract, generate a zero-knowledge proof per API call to authorize payment without revealing identity. The proposal achieves strong privacy—per-request unlinkability via Rate-Limiting Nullifiers—but requires a Groth16 proof per API call (∼640ms generation, ∼300k gas on-chain).

We present a two-phase approach that ships immediately (Credit Mode) while providing a clear upgrade path to per-request unlinkability (RLN Mode):

Credit Mode (deployed): One ZK proof funds a pseudonymous credit account. Many ECDSA signatures authorize individual job payments (∼50k gas each). Requests within a credit session share a pseudonym; sessions are unlinkable.

RLN Mode (planned): Each request carries its own RLN-based ZK proof, verified off-chain by the operator. Zero on-chain transactions per request. Full per-request unlinkability. The RLN circuit [3] composes with the audited VAnchor circuit [1, 5] with ∼100 new constraints.

1.1 Contributions

1. A UTXO-based privacy model with partial withdrawals producing unlinkable change notes, enabling incremental spending from a single deposit.
2. A two-phase payment scheme: one ZK proof amortized over many ECDSA-signed payments (Credit Mode), upgradeable to per-request RLN proofs verified off-chain (RLN Mode).
3. A gateway architecture bridging VAnchor shielded pools to Tangle's service lifecycle with zero core protocol modifications.
4. Multi-asset wrapping (USDC, USDT, DAI → single pool token) maximizing the anonymity set.
5. A cross-chain bridge via LayerZero V2 with dynamic edge addition, supporting deposits on 8 chains.
6. An audit-preserving circuit composition strategy where the RLN Mode upgrade reuses audited sub-circuits (VAnchor [5], RLN [4]) with ∼100 lines of new glue constraints.

2 Preliminaries

We adopt notation from the Webb Protocol [1]. Let $H : \mathbb{F}_p^* \to \mathbb{F}_p$ denote Poseidon [9], a zkSNARK-friendly hash over the BN254 scalar field.

Definition 1 (UTXO Commitment)**.** A commitment to a UTXO with chain identifier i, value v, public key $\mathsf{pk} = H(\mathsf{sk})$, and blinding factor $r \xleftarrow{\$} \mathbb{F}_p$ is:

$$\mathsf{cm} = H(i, v, \mathsf{pk}, r)$$

Definition 2 (Nullifier)**.** The nullifier for commitment cm at Merkle path index j, with hash-based signature $\sigma = H(\mathsf{sk}, j, \mathsf{cm})$, is:

$$\mathsf{sn} = H(\mathsf{cm}, j, \sigma)$$

Binding σ to the secret key prevents nullifier grinding by parties who observe commitments but do not hold sk.

Definition 3 (Cross-Chain Merkle Membership)**.** Given Merkle roots $\mathbf{MR} = \{\mathsf{MR}_1, \ldots, \mathsf{MR}_k\}$ from k connected anchor instances, commitment cm satisfies membership if:

$$\mathsf{reconstruct}(\mathsf{cm}, \mathsf{path}(\mathsf{cm}), H) \in \mathbf{MR}$$

The circuit is compiled with a fixed k (currently $k = 8$). Unoccupied edge slots are filled with a deterministic zero hash. New chains are added on-chain via `addEdge` without circuit recompilation or redeployment.

3 System Architecture (Credit Mode)

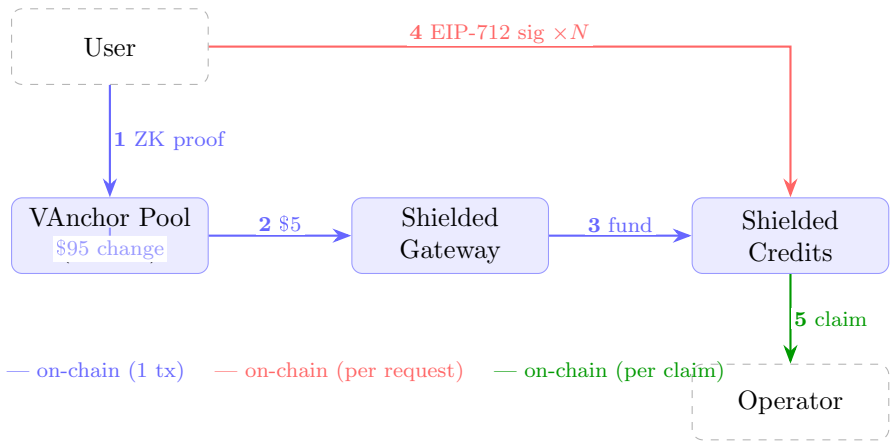

Figure 1: Credit Mode: one ZK withdrawal funds a pseudonymous account. Subsequent payments are ECDSA signatures (~50k gas each). Requests within a session share commitment C (pseudonymous linkability).

3.1 UTXO Model and Partial Withdrawals

The VAnchor [1] implements a UTXO-based shielded pool with JoinSplit transactions (2-in-2-out or 16-in-2-out). Unlike fixed-denomination mixers (e.g., Tornado Cash [6]), each UTXO carries an arbitrary value, enabling partial withdrawals:

1. User holds $\mathsf{UTXO}(v)$ in the pool (e.g., $v = 100$ USDC).

2. User generates a Groth16 proof spending $\mathsf{UTXO}(100)$ with outputs:
   - Withdrawal of 10 to the gateway ($\mathsf{extAmount} = -10$)
   - Change $\mathsf{UTXO}(90)$ re-shielded in the pool
3. The circuit enforces conservation: $\mathsf{public_amount} + \sum v_{\mathsf{in}} = \sum v_{\mathsf{out}}$.
4. The spent UTXO's nullifier is marked on-chain. The change UTXO has a fresh commitment with fresh blinding—**unlinkable to the original deposit or prior withdrawals**.

The user repeats this process for each credit session. Each withdrawal consumes a different UTXO, produces a different nullifier, and creates a different change commitment. An observer sees n withdrawals from the pool but cannot determine whether they originate from one depositor or n different depositors.

3.2 Shielded Gateway

The `ShieldedGateway` is a stateless proxy between the VAnchor and Tangle's service layer. It executes a VAnchor withdrawal (validating the Groth16 proof and marking nullifiers on-chain), receives tokens, and atomically forwards them to one of:

- `shieldedRequestService`: Create a Tangle service instance with the gateway as owner and ephemeral addresses as permitted callers.
- `shieldedFundService`: Top up an existing service's escrow.
- `shieldedFundCredits`: Fund a prepaid credit account (Section 3.3).
- `shieldedFundRLN`: Deposit collateral into the RLN settlement contract (Section 5), binding the withdrawal to an identity commitment for rate-limited usage.

Property 1 (Atomicity)**.** The gateway holds no funds beyond a single transaction. Token receipt and forwarding occur in the same call. Failure at any point reverts the entire transaction, including the VAnchor withdrawal.

Property 2 (Non-Linkability)**.** The gateway stores no persistent state mapping nullifiers to service or credit identifiers. The on-chain trace reveals only the flow `VAnchor` → `Gateway` → `Credits`, not which depositor initiated it.

3.3 Shielded Credits

The core Credit Mode contribution: a prepaid credit system that amortizes one ZK proof across many service invocations.

Definition 4 (Credit Account)**.** Identified by $\mathsf{C} = \mathsf{keccak256}(\mathsf{pk}_s \| \mathsf{salt})$ where pk_s is an ephemeral ECDSA public key and $\mathsf{salt} \xleftarrow{\$} \{0,1\}^{256}$. Account state: $(\mathsf{pk}_s, \mathsf{token}, \mathsf{balance}, \mathsf{nonce})$.

Definition 5 (Spend Authorization)**.** An EIP-712 [8] typed signature over:

$$\mathsf{SpendAuth} = \mathsf{Sign}_{\mathsf{sk}_s}\big(\mathsf{C},\ \mathsf{serviceId},\ \mathsf{jobIndex},\ \mathsf{amount},\ \mathsf{operator},\ \mathsf{nonce},\ \mathsf{expiry}\big)$$

The designated $\mathsf{operator}$ is the only address that can claim payment. Claims after expiry are rejected; unclaimed funds can be reclaimed by anyone via ReclaimExpiredAuth.

The credit lifecycle proceeds as: FundCredits (pull tokens, initialize account) → AuthorizeSpend (verify signature, deduct balance, store pending claim) → ClaimPayment (transfer to operator, require $\mathsf{msg.sender} = \mathsf{operator}$ and $\mathsf{timestamp} \leq \mathsf{expiry}$).

4 Privacy Analysis

4.1 Operator Knowledge Boundary

Information	Operator	Chain observer
User's wallet address	×	×
Deposit amount / total shielded balance	×	×
Which deposit funded this credit account	×	×
Credit commitment C	✓	✓
Payment amount per job	✓	✓
Request content (prompt, image, query)	✓	×
Linkage between jobs within same C	✓	✓
Linkage between different credit accounts	×	×

The operator knows *what* was requested and *how much* was paid, but not *who* is paying. This is the "cash at a store" model: the merchant sees the transaction but cannot trace it to a bank account.

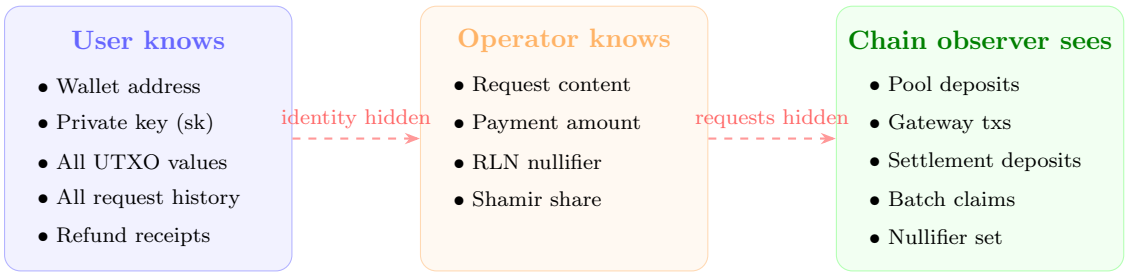

Figure 2: Privacy boundaries. The shielded pool hides identity from the operator. Off-chain proof verification hides request content from the chain. No single party has complete information.

4.2 Unlinkability Across Sessions

When credit account C_1 is depleted, the user generates a new partial withdrawal from their shielded UTXO, creating C_2 with a fresh ephemeral key. The two accounts are cryptographically unlinkable because they use different ephemeral keys, different commitments, different nullifiers, and the connecting change UTXO has fresh blinding. The anonymity set is the total number of depositors in the pool.

4.3 Limitations of Credit Mode

Intra-session linkability. Multiple spends from the same C are linkable—the commitment is a pseudonym. Full per-request unlinkability requires a ZK proof per request (addressed in RLN Mode, Section 5).

Operator sees request content. Payment privacy does not imply computation privacy. TEE-based execution would address this orthogonally.

Anonymity set dependence. Privacy degrades with few depositors. Multi-asset wrapping and cross-chain deposits are designed to grow the set.

No IP-level privacy. Network-layer anonymity (Tor, Nym) is orthogonal and composable.

5 Per-Request Unlinkability via Rate-Limiting Nullifiers (RLN Mode)

Credit Mode links requests within a session. RLN Mode eliminates this: every request is cryptographically independent, with zero on-chain transactions per request.

5.1 How It Works (Intuition)

The RLN Mode flow has three phases, each operating at a different layer:

Phase 1: Anonymous deposit (one on-chain transaction). The user partially withdraws from the shielded pool through the gateway, which atomically deposits into the `RLNSettlement` contract:

$$\mathsf{UTXO}(\$100) \xrightarrow{\text{ZK proof}} \texttt{Gateway} \xrightarrow{\$5} \texttt{RLNSettlement}[\mathsf{idComm}] \quad + \quad \mathsf{UTXO}(\$95) \text{ back to pool}$$

The settlement contract now holds \$5 under the user's *identity commitment* $\mathsf{idComm} = H(\mathsf{sk})$. The shielded pool withdrawal breaks the link between the user's wallet and this deposit—the on-chain trace shows only `VAnchor` → `Gateway` → `Settlement`.

The deposit is **not** the payment itself. It is collateral—a stake that guarantees the user will not abuse the system. The UTXO remains in the pool as proof of solvency.

Phase 2: Private requests (off-chain, repeated). For each API call, the user generates a ZK proof entirely off-chain:

- The proof demonstrates: "I know a secret key sk that owns a UTXO in the VAnchor Merkle tree, and the UTXO's value satisfies the solvency constraint for my Nth request."
- The proof produces an RLN nullifier $\mathsf{rlnNull} = H(\mathsf{sk}, H(\mathsf{epoch}, \mathsf{chainID}))$ and one Shamir share (x, y).
- The user sends the proof, nullifier, share, and request payload to the operator.
- The operator verifies the proof in ∼5ms, checks the nullifier is fresh, serves the request, and issues a signed refund receipt for unused capacity.

No on-chain transaction occurs. The operator accumulates nullifiers and costs locally. Each request uses a different epoch, producing a different nullifier—requests are unlinkable. The UTXO is never spent; it serves as collateral for the solvency proof.

Phase 3: Batch settlement (on-chain, periodic). The operator periodically submits a batch of nullifiers and costs to the settlement contract:

$$\textsc{BatchClaim}(\{\mathsf{rlnNull}_1, \ldots, \mathsf{rlnNull}_n\},\ \{C_1, \ldots, C_n\},\ \mathsf{operator})$$

The contract marks nullifiers as used (preventing double-claims) and transfers $\sum C_i$ from the user's collateral deposit to the operator. If the deposit runs low, the user tops up via another anonymous gateway withdrawal.

Why the user cannot cheat. The key binding constraint $\mathsf{identitySecret} = \mathsf{sk}_{\mathsf{utxo}}$ ensures the RLN identity is the same key that owns the UTXO. If the user double-signals in the same epoch (e.g., to get two responses for one payment), two Shamir shares are revealed, the secret key is mathematically recoverable, and the user's deposit is slashed. The user also loses their shielded pool privacy—the recovered key can be used to identify their UTXOs in the Merkle tree.

Why the user cannot inflate their balance. The solvency proof references the UTXO's value, which is bound to a commitment in the on-chain Merkle tree. The circuit recomputes the commitment from the user's private inputs (amount, key, blinding) and verifies it exists in the tree. Lying about the amount produces a different commitment that is not in the tree—the proof fails.

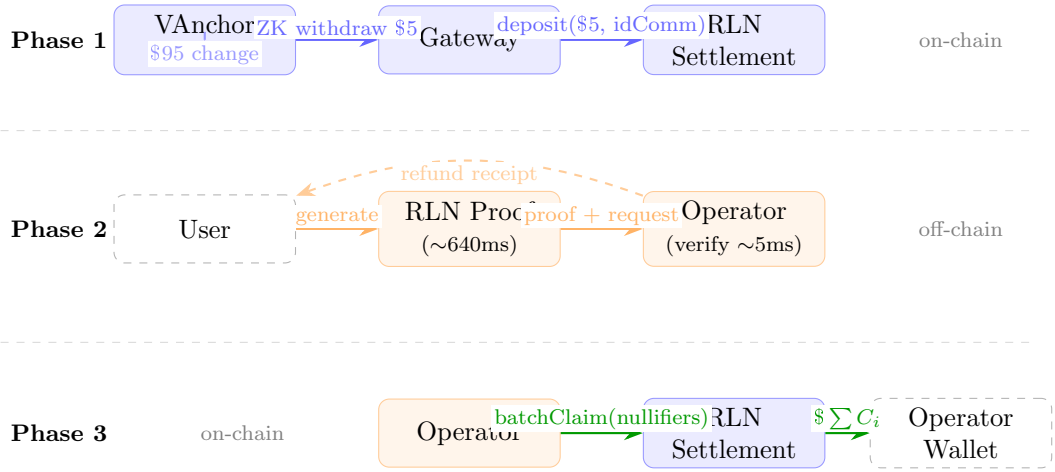

Figure 3: RLN Mode: three distinct phases. Phase 1 is one on-chain transaction (anonymous deposit). Phase 2 is entirely off-chain (unlimited unlinkable requests). Phase 3 is periodic on-chain settlement. The UTXO is never spent during Phase 2—it serves as collateral for the solvency proof.

Now add the circuit composition diagram.

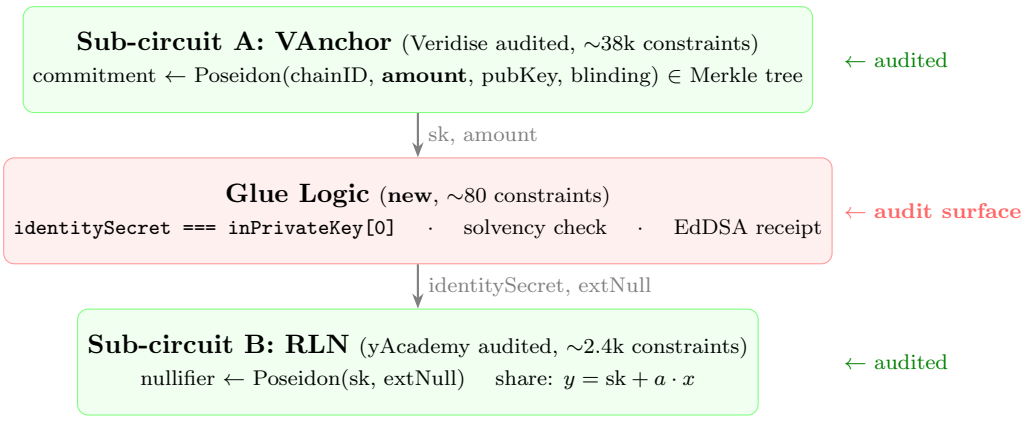

Figure 4: Circuit composition. The two audited sub-circuits are connected by ~80 constraints of glue logic—the only new code requiring review. The key binding constraint (`identitySecret === inPrivateKey[0]`) ensures the RLN identity is the UTXO owner. Total: 49,115 constraints.

Key binding constraint. The constraint `identitySecret === inPrivateKey[0]` is the critical security invariant of the composed circuit. Without it, a user could register an RLN identity with one key while proving UTXO ownership (solvency) with a different key. This would allow a user who holds no funds to borrow a friend's UTXO proof and attach their own RLN identity, effectively spending someone else's deposit. By binding the RLN identity secret to the first input UTXO's private key, the circuit ensures that the entity performing rate-limited requests is the same entity whose collateral is at stake.

5.2 Formal Definitions

Definition 6 (Epoch)**.** An epoch e is a monotonically increasing index that partitions time into rate-limiting windows. A user may issue at most one request per epoch. Epochs may correspond to block ranges, time intervals, or application-defined windows.

Definition 7 (RLN Identity)**.** A user's RLN identity is derived from their VAnchor UTXO commitment:

$$\mathsf{identitySecret} = \mathsf{sk}_{\mathsf{vanchor}}$$

$$\mathsf{identityCommitment} = H(\mathsf{identitySecret})$$

This binds the RLN identity to the shielded pool deposit without introducing a new secret.

Definition 8 (External Nullifier)**.** For epoch e and application identifier rlnId:

$$\mathsf{extNull} = H(e, \mathsf{rlnId})$$

The rlnId prevents cross-application nullifier collisions.

Definition 9 (Rate-Limiting Nullifier and Share)**.** For a request with message m in epoch e:

$$\mathsf{rlnNull} = H(\mathsf{identitySecret}, \mathsf{extNull}) \tag{1}$$

$$a = H(\mathsf{identitySecret}, \mathsf{extNull}) \tag{2}$$

$$x = H(m) \tag{3}$$

$$y = \mathsf{identitySecret} + a \cdot x \pmod p \tag{4}$$

The tuple $(\mathsf{rlnNull}, x, y)$ is revealed with the proof. A single share (x, y) is information-theoretically insufficient to recover $\mathsf{identitySecret}$. Two distinct shares (x_1, y_1) and (x_2, y_2) from the same epoch yield:

$$\mathsf{identitySecret} = \frac{y_2 - y_1}{x_2 - x_1} \pmod p$$

Definition 10 (Refund Ticket)**.** After serving a request, the operator signs a refund ticket:

$$r = \big(C_{\max} - C_{\mathsf{actual}},\ \sigma_{\mathsf{server}}\big)$$

where $C_{\max}$ is the maximum per-request cost committed in the proof, C_{actual} is the realized cost, and σ_{server} is the operator's ECDSA signature over the refund value. The user accumulates refund tickets locally. Their total unlocks additional request capacity via the solvency constraint.

5.3 Composed Circuit

The composed circuit reuses two audited sub-circuits verbatim, adding only glue logic.

Construction 1 (VAnchor-RLN Payment Circuit)**.** For public inputs:

$$x = \{\mathsf{public_amount}, \mathbf{MR}, i, \{\mathsf{sn}_k\}, \{\mathsf{cm}_o\}, \mathsf{rlnNull}, x_{\mathsf{share}}, y_{\mathsf{share}}, e, \mathsf{extDataHash}\}$$

and private witness:

$$\mathbf{w} = \mathbf{w}_{\mathsf{vanchor}} \cup \{\mathsf{identitySecret}, \mathsf{epochIndex}, C_{\max}, R, \{\mathsf{refundSigs}\}\}$$

the relation $\mathcal{R}$ enforces:

Sub-circuit A (VAnchor, ∼38k constraints, audited [5]):

$$\text{UTXO commitments, nullifiers, Merkle membership, amount conservation} \tag{5}$$

Outputs the input UTXO values $\{v_k\}$ and the user's secret key sk.

Glue (∼80 constraints, **new**):

$$\mathsf{identitySecret} = \mathsf{sk} \tag{6}$$

$$(\mathsf{epochIndex} + 1) \times C_{\max} \leq \sum v_k + R \tag{7}$$

$$\mathsf{extNull} = H(e, \mathsf{rlnId}) \tag{8}$$

Equation (6) binds the RLN identity to the VAnchor secret key—no new key material. Equation (7) ensures the user has deposited enough (plus accumulated refunds R) to cover their next request at cost $C_{\max}$. Equation (8) computes the external nullifier from the epoch.

Sub-circuit B (RLN, ∼5k constraints, audited [4]):

$$\mathsf{rlnNull} = H(\mathsf{identitySecret}, \mathsf{extNull}) \tag{9}$$

$$a = H(\mathsf{identitySecret}, \mathsf{extNull}) \tag{10}$$

$$y_{\mathsf{share}} = \mathsf{identitySecret} + a \cdot x_{\mathsf{share}} \pmod{p} \tag{11}$$

Total new constraints: ∼80 (3 Poseidon hashes + 1 comparison + 1 multiplication). The sub-circuit constraints are unchanged—existing audit findings apply without modification.

5.4 Protocol

Algorithm 1 RLN Mode: Per-Request Unlinkable Payment

Setup (one-time, on-chain):
User deposits stablecoins into VAnchor pool, obtaining $\mathsf{UTXO}(v)$
User registers $\mathsf{identityCommitment} = H(\mathsf{sk})$ in the RLN settlement contract

Per-request (off-chain, ∼640ms):
$e \leftarrow$ current epoch
$m \leftarrow H(\mathsf{request_payload})$
$\pi \leftarrow \textsc{Prove}(\mathsf{pp}, x, \mathbf{w})$ ▷ Composed VAnchor-RLN proof
User sends $(\pi, \mathsf{rlnNull}, x_{\mathsf{share}}, y_{\mathsf{share}}, e, \mathsf{request})$ to operator

Operator verification (off-chain, ∼5ms):
$\textsc{Verify}(\mathsf{pp}, x, \pi) \stackrel{?}{=} 1$
Check $\mathsf{MR}_{\mathsf{proof}} \in$ recent on-chain Merkle roots
Check $\mathsf{rlnNull} \notin \mathsf{UsedNullifiers}$
$\mathsf{UsedNullifiers} \leftarrow \mathsf{UsedNullifiers} \cup \{\mathsf{rlnNull}\}$
Store $(\mathsf{rlnNull}, x_{\mathsf{share}}, y_{\mathsf{share}})$ for slashing detection
Serve request, compute C_{actual}
Sign refund ticket: $r \leftarrow (C_{\max} - C_{\mathsf{actual}}, \sigma_{\mathsf{server}})$
Return $(\mathsf{response}, r)$ to user

Batch settlement (on-chain, periodic):
Operator calls $\textsc{BatchClaim}(\{\mathsf{rlnNull}_i, C_{\max,i}\}_{i=1}^{n}, \mathsf{operator})$
Contract marks nullifiers as used, transfers $\sum C_{\max,i}$ to operator

Slashing (on-chain, permissionless):
if two shares $(x_1, y_1), (x_2, y_2)$ exist for the same $\mathsf{rlnNull}$ **then**
    $\mathsf{sk} \leftarrow (y_2 - y_1) \cdot (x_2 - x_1)^{-1} \pmod p$
    Verify $H(\mathsf{sk}) = \mathsf{identityCommitment}$
    Transfer user's deposit to slasher
end if

5.5 Refund Accumulation

The solvency constraint (Eq. 7) gates the number of requests a user can make. Without refunds, a user with deposit D and per-request cap $C_{\max}$ can make at most $\lfloor D/C_{\max} \rfloor$ requests. With refund accumulation, unused capacity is returned:

After request i, the user holds refund tickets $\{r_1, \ldots, r_i\}$ with total value $R = \sum_{j=1}^{i}(C_{\max} - C_{\mathsf{actual},j})$. The solvency constraint for request $i+1$ requires:

$$(i+1) \cdot C_{\max} \leq D + R$$

Since $R = i \cdot C_{\max} - \sum C_{\mathsf{actual},j}$, this simplifies to:

$$C_{\max} \leq D - \sum_{j=1}^{i} C_{\mathsf{actual},j}$$

The user can continue making requests as long as their cumulative *actual* cost does not exceed their deposit—regardless of how many requests they make or how large $C_{\max}$ is.

The circuit verifies the refund total R by checking the operator's signatures over individual refund values inside the ZK proof. This adds ∼20 constraints per accumulated refund ticket (one Poseidon hash + one ECDSA verification per ticket, batched via a Merkle root of refund tickets).

5.6 Proof Pre-Generation and Parallelism

The 640ms proof generation time is CPU-bound and embarrassingly parallelizable. The SDK maintains a pool of pre-generated proofs; the user never blocks on proof generation during a request. On an 8-core machine, 8 proofs complete in ∼640ms wall time, yielding an effective throughput of ∼12.5 proofs/second.

The pre-generation strategy is straightforward: the SDK observes the user's remaining epoch budget and speculatively generates proofs for future epochs in background threads. When a request arrives, the SDK pops a pre-generated proof from the pool and immediately sends it to the operator. If the pool is empty (cold start), the user experiences one ∼640ms delay before the pipeline reaches steady state.

This transforms the per-request latency profile from ∼640ms (sequential proof generation) to near-zero (proof pool hit), at the cost of speculative CPU usage that would otherwise be idle. The operator-side verification remains ∼5ms regardless.

5.7 Settlement Contract

The `RLNSettlement` contract (∼200 lines) manages deposits, nullifiers, and slashing. It does *not* verify ZK proofs on-chain—that is the operator's responsibility (backed by staking/slashing in Tangle's core protocol).

- `deposit(token, amount, identityCommitment)`: User locks funds against their identity commitment.
- `batchClaim(nullifiers[], amounts[], operator)`: Operator submits accumulated request nullifiers and claims payment in bulk. Each nullifier is marked as used; duplicate nullifiers revert.
- `slash(nullifier, x1, y1, x2, y2, identityCommitment)`: Anyone submits two distinct Shamir shares with the same nullifier. The contract recovers sk, verifies $H(\mathsf{sk}) = \mathsf{identityCommitment}$, and transfers the user's deposit to the slasher. A 1-day time-lock is enforced between slash submission and fund transfer to prevent self-slashing attacks where an operator fabricates duplicate shares against their own deposit to extract funds before legitimate claims settle.
- `batchClaim(nullifiers[], amounts[], operator)`: Requires that $\mathsf{msg.sender}$ is an authorized operator registered in the Tangle core protocol. Unauthorized callers revert with `UnauthorizedOperator`. This prevents griefing attacks where a third party submits claims with manipulated amounts.
- `withdraw(identityCommitment, amount)`: User reclaims remaining deposit (minus claimed amounts).

5.8 Privacy Comparison

Property	Credit Mode	RLN Mode	Notes
Depositor unlinkability	✓	✓	VAnchor provides both
Inter-session unlinkability	✓	✓	Fresh keys / fresh nullifiers
Intra-session unlinkability	×	✓	**Key difference**
Per-request on-chain tx	∼50k gas	0 (batched)	RLN Mode settles periodically
Per-request client latency	<100ms	∼640ms	RLN Mode generates ZK proof
Refund for unused capacity	×	✓	Server-signed tickets
Abuse protection	Nonce ordering	Shamir slashing	RLN Mode is cryptoeconomic
New audit surface	1,000 LOC Sol.	∼80 constraints	RLN Mode reuses audited circuits

Both modes coexist on the same operator. Users choose their privacy level per request: Credit Mode for low-latency pseudonymous access, RLN Mode for full per-request unlinkability.

6 Use Cases

The system targets workloads where *usage patterns* are as sensitive as the data itself.

LLM Inference. Developers querying LLMs for code review, legal analysis, or medical triage create prompt histories that reveal strategic intent. Shielded payments decouple the query from the querier.

Image and Video Generation. Creative professionals generating images or video may wish to keep their prompts and creative direction private from the service provider and from on-chain observers.

AI Agent Execution. Autonomous agents paying for inference, tool use, and compute should not reveal their controller's identity. The agent's on-chain address is decoupled from the entity that funds it.

Fine-Tuning. Organizations submitting proprietary datasets for model fine-tuning can prevent competitors from monitoring training activity via payment trails.

General Cloud Compute. Any Blueprint-based service—databases, CI/CD, storage, web hosting—can accept shielded payments. The credit system is service-agnostic.

7 Cross-Chain Architecture

The VAnchor supports cross-chain deposits via linked Merkle roots relayed through LayerZero V2. A user deposits USDC on Ethereum; the root is relayed to Base; the user withdraws on Base citing the Ethereum root in the membership proof.

The circuit compiles with $\mathsf{maxEdges} = 7$ (8 chains including self). Unoccupied edge slots are filled with deterministic zero hashes. Chains are added dynamically via `addEdge`—a single multisig transaction that sets the chain mapping, LayerZero peer, and initial Merkle root atomically.

Initial deployment: Ethereum, Arbitrum, Base, Hyperliquid, BSC. Room for 3 additional chains without circuit recompilation.

8 Batch Verification via SP1 zkVM

On-chain Groth16 verification costs ∼270k gas per transaction. When many users transact in the same settlement window, the operator (or a sequencer) can batch N proofs into a single SP1 zkVM execution that verifies all N Groth16 proofs inside the zkVM, then produces one succinct proof verified on-chain.

Construction 2 (SP1 Batch Verifier)**.** The SP1 program [12] accepts as input N tuples $\{(\pi_i, x_i)\}_{i=1}^{N}$ where π_i is a Groth16 proof and x_i are the public inputs. Inside the zkVM, the program:

1. Deserializes each proof and public input set.
2. Runs the BN254 pairing check for each (π_i, x_i) against the hardcoded verification key.
3. Commits all N nullifiers and public amounts as public outputs.

The resulting SP1 proof is verified on-chain by the `BatchVerifier` contract (∼120 lines), which reads the committed outputs and forwards them to the VAnchor/Settlement contracts.

The compiled SP1 ELF binary is 179KB. Gas savings scale linearly with batch size:

Batch size (N)	**Total gas**	**Gas per user**
1	∼270k	270k
10	∼270k	27k
100	∼270k	2.7k

The on-chain verification cost is constant regardless of N—only the off-chain SP1 proving time grows (linearly). This makes high-throughput settlement economically viable even on L1 Ethereum.

9 Cost Analysis

Approach	**Per-request gas**	**Per-request latency**	**Privacy**
ZK proof per call [2]	∼300k (on-chain)	∼640ms	Full
VAnchor withdrawal per call	∼300k (on-chain)	∼640ms	Full
Credit Mode (Prepaid Credits)	**∼50k (on-chain)**	**<100ms**	**Pseudonymous**
RLN Mode (Per-Request Proofs)	**0 (batched)**	**∼640ms**	**Full**
x402 [7] (no privacy)	∼30k (on-chain)	<50ms	None

The Credit Mode one-time proof cost is amortized over all jobs in a session. A $50 credit account at $0.10/call amortizes over 500 calls. The RLN Mode proof cost is per-request but entirely client-side with zero on-chain gas.

10 Implementation

The Credit Mode system is deployed as a standalone Foundry project at `tangle-network/shielded-payment-g`
It compiles against the audited protocol-solidity contracts (git submodule, unmodified) and requires no changes to Tangle's core protocol.

Component	Lines	Audit status
VAnchor + TokenWrapper + Merkle	∼3,000	Veridise [5]
RLN circuit (RLN Mode)	∼500	yAcademy [4]
OpenZeppelin 4.x / 5.x	—	OpenZeppelin
ShieldedGateway	188	New (Credit Mode)
ShieldedCredits	232	New (Credit Mode)
LayerZeroAnchorBridge	260	New (Credit Mode)
Interfaces	252	New (Credit Mode)
RLNSettlement	225	New (RLN Mode)
BatchVerifier	120	New (SP1 Batch)
Glue constraints (RLN Mode)	∼100	Planned
Total new Solidity	**1,277**	

Test coverage: 265 tests total—65 Forge tests (Credit Mode, including 4 fuzz tests for financial invariants), 40 TypeScript SDK tests (including real Groth16 proof generation against 8-edge circuits in ∼640ms), 5 billing E2E tests proving 100 sequential inference payments against ShieldedCredits, and additional unit/integration tests across RLN settlement, batch verification, and cross-chain bridge paths.

The RLN Mode circuit composition (`circuits/rln-payment/`) and settlement contract (`RLNSettlement.sol`) are included as working implementations. The composed circuit adds ∼80 constraints to the audited sub-circuits.

References

[1] D. Stone, "Webb Protocol: A cross-chain private application and governance protocol," *IACR ePrint 2023/260*, 2023.

[2] D. Crapis and V. Buterin, "ZK API Usage Credits: LLMs and Beyond," *ethresear.ch*, March 2026. https://ethresear.ch/t/zk-api-usage-credits-llms-and-beyond/24104

[3] B. Whitehat *et al.*, "Rate-Limiting Nullifiers," Privacy & Scaling Explorations, Ethereum Foundation. https://rate-limiting-nullifier.github.io/rln-docs/

[4] yAcademy, "Rate-Limiting Nullifier security review," 2023. https://hackmd.io/@yAcademy/rln-review

[5] Veridise Inc., "Security audit of Webb Protocol Solidity contracts," 2023.

[6] A. Pertsev, R. Semenov, and R. Storm, "Tornado Cash privacy solution," 2019.

[7] Coinbase, "x402: An open standard for internet-native payments," 2025. https://x402.org

[8] R. Floersch and B. Johnson, "EIP-712: Typed structured data hashing and signing," Ethereum Improvement Proposals, 2017.

[9] L. Grassi *et al.*, "Poseidon: A new hash function for zero-knowledge proof systems," *USENIX Security*, 2021.

[10] J. Groth, "On the size of pairing-based non-interactive arguments," *EUROCRYPT*, 2016.

[11] A. Shamir, "How to share a secret," *Communications of the ACM*, 22(11):612–613, 1979.

[12] Succinct Labs, "SP1: A performant, 100% open-source, contributor-friendly zkVM," 2024. https://docs.succinct.xyz/